

UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE MATEMATICHE,
INFORMATICHE E FISICHE

IBML - Internet of Things, Big Data, Machine Learning



Internet of Things Progetto Pet Tracker

Autori

Andrea Gioia, 169484
Alessio Nicodemo, 166972
Mattia Nadal, 167372
Francesco Sabot, 167339

169484@spes.uniud.it
166972@spes.uniud.it
167372@spes.uniud.it
167339@spes.uniud.it

Sommario

Il progetto Pet-Tracker propone la realizzazione di un sistema IoT per il tracciamento in tempo reale di animali domestici di taglia media o superiore, scelta dettata dal peso del dispositivo. La soluzione integra un dispositivo embedded basato su ESP32-S3 con connettività LTE e modulo GNSS, in grado di acquisire e trasmettere dati di posizione, stato della batteria e attività motoria verso un backend remoto.

L'architettura segue il modello IoT World Forum (IoTWF) e adotta tecniche di ottimizzazione energetica, tra cui modalità Deep Sleep e strategie di Hot Start per il GPS. I dati vengono elaborati e resi disponibili tramite API REST e visualizzati attraverso un'applicazione mobile dedicata.

L'obiettivo principale è garantire un sistema affidabile, a basso consumo energetico e scalabile per il monitoraggio continuo dell'animale.

Indice

1	Architettura del sistema	4
1.1	Mappatura sul modello IoTWF	4
2	Hardware	4
2.1	LiLyGo T-SIM7670G-S3	4
2.2	Antenna GPS e LTE	4
2.3	Batteria e Power Management	4
2.4	BMA400 CJMCU-400	5
3	Firmware: Logica di funzionamento	5
3.1	Gestione della Persistenza e Hot Start	5
3.2	Protocollo di Comunicazione HTTPS	5
3.3	Risparmio Energetico e Deep Sleep	6
3.4	Monitoraggio Hardware e Batteria	6
4	Back-end e Database	6
4.1	Architettura di Hosting e Network Exposure	6
4.2	Sistema di Backup e Data Retention	7
4.3	Monitoraggio e Business Continuity	7
4.4	Struttura dei dati e pulizia automatica	7
4.5	Interfaccia API e API Rules	7
4.5.1	API Rules e Sicurezza	8
4.6	Smistamento e Normalizzazione	8
4.6.1	Ottimizzazione delle query: lettura centralizzata della board	8
4.6.2	Meccanismo di Trip Hold (filtro falsi positivi)	8
4.6.3	Funzionamento del Trigger onRecordAfterCreateSuccess	9
4.6.4	Macchina a stati delle attività	9
4.6.5	Gestione del Risveglio e Deduplicazione delle Sessioni	10
4.6.6	Watchdog di Inattività e Split Notturmo	10
4.6.7	Vantaggi Architettureali e Corrispondenza IoTWF	11
5	Sistema di Notifiche Push	11
5.1	Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase	11
5.2	Architettura del FCM Bridge	12
5.3	Logica di Invio e Trigger	12
6	Front-end e Applicazione Mobile	12
6.1	Architettura a livelli	13
6.2	Autenticazione e persistenza della sessione	13
6.3	Splash Screen e Caricamento Parallelo	13
6.4	Comunicazione in Tempo Reale	13
6.5	Geofencing e Geocoding	14
6.6	Permessi e Notifiche Push	14
6.7	Funzionalità Principali	15
7	Test e Risultati	15

8 Limiti del Sistema	15
9 Conclusioni	16
Riferimenti bibliografici	17

1 Architettura del sistema

L'architettura del sistema Pet-Tracker segue il **Modello a 7 livelli dell'IoT World Forum (IoTWF)**, garantendo una transizione fluida dal dato fisico all'informazione per l'utente.

1.1 Mappatura sul modello IOTWF

- **Livello 1 (Physical Devices):** Rappresentato dalla board LilyGo e dai sensori (GPS, Accelerometro).
- **Livello 2 (Connectivity):** Gestito tramite il modem LTE e protocolli di trasporto sicuri.
- **Livello 3 (Edge Computing):** Elaborazione locale sul microcontrollore per il filtraggio delle stringhe NMEA e il calcolo dei passi.
- **Livello 4 (Data Accumulation):** Persistenza dei dati sul server PocketBase (database SQLite).
- **Livello 5 (Data Abstraction):** Esposizione dei dati tramite API REST strutturate in formato JSON.
- **Livello 6 (Application):** L'applicazione mobile dedicata che trasforma le coordinate in posizioni su mappa.
- **Livello 7 (Collaboration and Processes):** L'interazione del proprietario con il sistema per il monitoraggio dell'animale.

2 Hardware

2.1 LiLyGo T-SIM7670G-S3

Il dispositivo utilizza una board LiLyGo T-SIM7670G-S3 [1] che si basa sul modulo **ESP32-S3** dual-core, che gestisce sia la logica applicativa che la comunicazione con il modem SIM7670G tramite interfaccia UART.

2.2 Antenna GPS e LTE

Il sistema utilizza un'antenna flessibile LTE [2] per la connettività dati e un'antenna ceramica attiva [3] per il ricevitore GNSS integrato, essenziale per la precisione della localizzazione outdoor.

2.3 Batteria e Power Management

Viene impiegata una batteria ricaricabile 3000mAh 30A Litio-Ion ad alta capacità. Il monitoraggio avviene tramite un partitore di tensione collegato al pin `ADC_BAT` (GPIO 4), permettendo di calcolare la percentuale di carica residua.

2.4 BMA400 CJMCU-400

L'accelerometro usato è un **BMA400** [4] integrato nella breadboard **CJMCU-400**, che viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio della salute dell'animale.

3 Firmware: Logica di funzionamento

Il firmware, sviluppato in ambiente **Arduino** per il SoC **ESP32-S3**, gestisce l'acquisizione dei dati dai sensori, il controllo del modem **LTE** e l'ottimizzazione energetica tramite una macchina a stati finiti.

3.1 Gestione della Persistenza e Hot Start

Per ridurre drasticamente il *Time To First Fix* (TTFF) del modulo GNSS, il firmware sfrutta la memoria **RTC (Real-Time Clock)** dell'ESP32. Questa memoria permette di mantenere i dati vitali anche durante i cicli di *Deep Sleep*. Le variabili critiche, come l'ultima posizione valida (`lastLat`, `lastLon`) e i timestamp (`lastGpsDate`, `lastGpsTime`), vengono dichiarate con l'attributo `RTC_DATA_ATTR`.

Al risveglio del dispositivo, la funzione `iniettaGps()` esegue le seguenti operazioni:

- Recupera le coordinate geografiche e temporali salvate nel ciclo precedente.
- Inietta la posizione nota nel modulo GNSS tramite il comando `AT+CGNSSPOS`.
- Sincronizza l'orario del modem con il comando `AT+CGNSSTIME`.

Questo meccanismo abilita la modalità **Hot Start**, che minimizza il tempo di ricerca dei satelliti e, di conseguenza, il consumo energetico.

3.2 Protocollo di Comunicazione HTTPS

La trasmissione dei dati al backend **PocketBase** è affidata a una serie di comandi `AT` inviati via seriale al modem **SIM7670G**. Il firmware incapsula le informazioni in un oggetto **JSON** strutturato, inviato tramite una richiesta **HTTPS POST**. Il payload include:

- `board_id`: un identificativo univoco derivato dall'IMEI del modem;
- `geo`: un oggetto con latitudine e longitudine per la gestione cartografica;
- `battery / battery_percent / charging`: telemetria sullo stato di carica della batteria;
- `steps`: conteggio dei passi rilevato dall'accelerometro;
- `sleep`: flag booleano che segnala al server l'imminente entrata in stato di inattività;
- `trip`: flag booleano che segnala al server che l'animale si trova su un veicolo in movimento. Quando attivo, ha priorità assoluta sulla logica di geofencing e sopprime il *Deep Sleep* per garantire la continuità del tracciamento.

3.3 Risparmio Energetico e Deep Sleep

Il sistema è progettato per massimizzare l'autonomia della batteria da 3000mAh. La logica di risparmio energetico è coordinata con l'accelerometro **BMA400** e tiene conto dello stato di viaggio:

1. **Rilevamento Attività:** Il firmware monitora costantemente il numero di passi. Ogni nuovo movimento resetta un timer di inattività (`lastActivityTime`).
2. **Modalità Viaggio:** Se il flag `trip` è attivo, il Deep Sleep viene soppresso indipendentemente dall'inattività rilevata dall'accelerometro. Il dispositivo continua a trasmettere posizione e telemetria a intervalli regolari per tutto il tempo in cui è in viaggio.
3. **Invio Finale:** Se non viene rilevata attività per un tempo superiore a `SLEEP_TIMEOUT` (30 secondi) e il flag `trip` non è attivo, il sistema invia un ultimo pacchetto dati aggiornando il flag `sleep` a `true`.
4. **Ibernazione:** Il microcontrollore entra in *Deep Sleep*, spegnendo i moduli radio. Il risveglio è gestito tramite interrupt hardware sul pin `GPIO_5`, generato dal BMA400 solo al rilevamento di un nuovo movimento.

3.4 Monitoraggio Hardware e Batteria

Il firmware gestisce in modo intelligente l'alimentazione per evitare letture errate o consumi parassiti:

- **Controllo ADC:** Il circuito del partitore di tensione per la batteria viene attivato solo durante la lettura tramite il pin `BAT_ADC_EN`, garantendo la precisione del voltaggio acquisito dal pin `ADC_BAT`.
- **Stato USB:** Attraverso la lettura dei registri hardware dell'ESP32, il sistema rileva se il dispositivo è collegato a una fonte di ricarica USB, settando dinamicamente il campo `charging` nel pacchetto dati.

4 Back-end e Database

Il centro nevralgico della gestione dati è basato su **PocketBase** [5], una soluzione open-source che integra un database embedded SQLite con un'interfaccia API REST.

4.1 Architettura di Hosting e Network Exposure

L'istanza di PocketBase è stata ospitata su una workstation locale e data la natura del sistema IoT, che richiede la raggiungibilità del backend da parte del dispositivo (tramite rete LTE) e dell'applicazione mobile (tramite rete dati/Wi-Fi), si è reso necessario esporre il servizio locale su rete pubblica. Il problema del *NAT Traversal* e dell'assenza di un indirizzo IP pubblico statico sulla rete locale è stato risolto mediante l'impiego di **ngrok** [6]. Questa tecnologia di *tunneling* crea un tunnel sicuro (HTTPS) tra l'endpoint pubblico di ngrok e la porta locale su cui è in esecuzione il server. Tale URL funge da *Reverse Proxy*, reindirizzando il traffico HTTPS in ingresso verso l'istanza locale di PocketBase,

garantendo così la persistenza dei dati e l'accessibilità dei servizi in tempo reale ovunque sia presente una connessione internet.

4.2 Sistema di Backup e Data Retention

Per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato implementato un sistema di backup automatizzato. Il processo utilizza l'utility **rclone** [7], uno strumento a riga di comando per la gestione del cloud storage, configurato per eseguire la sincronizzazione della directory dei dati di PocketBase verso un bucket remoto su **Backblaze B2** [8]. L'automazione è gestita tramite un *cron job* che esegue il backup in modo incrementale ogni primo giorno del mese.

4.3 Monitoraggio e Business Continuity

Per garantire l'affidabilità dell'intero ecosistema, è stato implementato un sistema di monitoraggio esterno tramite **UptimeRobot** [9]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il monitoraggio interroga l'endpoint HTTPS a intervalli regolari, inviando notifiche immediate in caso di downtime.

4.4 Struttura dei dati e pulizia automatica

I dati ricevuti dalla board transitano nella collezione `data_sent_raw`, che funge da punto di ingresso grezzo. Ogni record viene immediatamente smistato nelle collezioni specializzate dall'hook server-side e successivamente eliminato nel blocco `finally` dell'hook stesso. Questo approccio garantisce che il database non accumuli dati ridondanti, mantenendo al contempo un audit trail completo nelle collezioni di destinazione. La struttura del record prevede i seguenti campi:

- `timestamp`: riferimento temporale dell'acquisizione.
- `lat / lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery`: stato di carica della batteria (espresso in voltaggio);
- `battery_percent`: stato di carica della batteria (espresso in percentuale);
- `charging`: booleano che indica se la batteria è in corso di ricarica;
- `steps`: conteggio dei passi rilevato dall'accelerometro BMA400;
- `sleep`: booleano che indica se il dispositivo è in stato di inattività prolungata;
- `trip`: booleano che indica se l'animale si trova su un veicolo in movimento.

4.5 Interfaccia API e API Rules

L'accesso ai dati memorizzati nel backend avviene esclusivamente tramite un'interfaccia **RESTful API** fornita nativamente da PocketBase. Questo approccio permette di separare nettamente il livello di persistenza (Database) dal livello di fruizione (Applicazione Mobile).

4.5.1 API Rules e Sicurezza

Per garantire l'integrità e la riservatezza dei dati, sono state configurate specifiche **API Rules** basate su espressioni booleane:

- **List/View:** Visualizzazione permessa solo agli utenti autenticati.
- **Create:** Abilitata per il dispositivo LilyGo tramite autenticazione dedicata.
- **Update/Delete:** Ristrette ai soli amministratori del sistema.

4.6 Logica di Smistamento e Normalizzazione (Server-side Hooks)

Per ottimizzare la gestione dei dati e garantire la separazione delle responsabilità, è stata implementata una logica di smistamento automatico lato server tramite i *JavaScript Event Hooks* di PocketBase. L'hook `onRecordAfterCreateSuccess` si attiva immediatamente dopo il salvataggio di ogni pacchetto grezzo, smista i dati nelle collezioni specializzate e infine elimina il record da `data_sent_raw` nel blocco `finally`, garantendo la pulizia anche in caso di errori parziali.

4.6.1 Ottimizzazione delle query: lettura centralizzata della board

Il record board viene letto una sola volta all'inizio dell'hook tramite `getBoardRecord` e passato come parametro a tutte le funzioni che ne hanno bisogno (`computeStatus`, `saveBattery`, `notifyBoardUsers`). Nella versione precedente ogni funzione eseguiva una query indipendente, producendo 3-4 letture ridondanti per ogni pacchetto ricevuto. La versione attuale elimina completamente questo overhead.

4.6.2 Meccanismo di Trip Hold (filtro falsi positivi)

Il dispositivo può inviare occasionalmente pacchetti con `trip=true` per errore hardware. Per evitare cambi di stato non voluti, è stato implementato un meccanismo di conferma a due pacchetti denominato **Trip Hold**, basato sul campo JSON `pending_trip` nella collezione `boards`.

Il flusso opera come segue:

1. **Primo pacchetto `trip=true` (Hold):** i dati del pacchetto vengono salvati su `board.pending_trip`. Viene processata solo la batteria. La posizione non viene salvata e il blocco `activity` non viene raggiunto (il codice esce con `return`). La sessione corrente rimane invariata.
2. **Secondo pacchetto `trip=true` (Conferma):** il viaggio è reale. Viene processata la batteria del pacchetto `pending`, viene aperta la nuova `activity` con stato `"v"`, e le posizioni di entrambi i pacchetti vengono salvate agganciandole alla nuova sessione.
3. **Pacchetto `trip=false` (Falso positivo):** il `pending` viene cancellato. I dati GPS del pacchetto in `hold` vengono tenuti in una variabile temporanea (`pendingToSave`) e la relativa posizione viene salvata nel blocco finale, agganciata all'`activity` corrente determinata dalla logica normale – non prima.
4. **Pacchetto `sleep=true` durante il hold:** il `pending` viene cancellato immediatamente, ma il codice *non* esce con `return`: prosegue con il flusso `sleep` normale. Questo evita che un `trip=true` non confermato influenzi lo stato dell'animale.

5. **Watchdog durante il hold:** se il dispositivo va offline, il watchdog cancella il pending insieme alla sessione.

La proprietà chiave di questo meccanismo è che il pending non può mai influenzare la logica di riattivazione delle activity: il blocco hold esce con `return` prima del blocco activity (primo pacchetto), oppure il pending è già stato azzerato prima che il blocco activity venga raggiunto (tutti gli altri casi).

4.6.3 Funzionamento del Trigger onRecordAfterCreateSuccess

Il trigger elabora ogni pacchetto in cinque fasi sequenziali:

- **Trip Hold:** verifica e gestisce il meccanismo di conferma del viaggio descritto sopra.
- **Batteria:** salva il record `battery_data` e controlla se inviare notifiche di cambio soglia. Viene processata per tutti i casi tranne il primo `trip=true` (già gestito nel blocco hold prima del `return`).
- **Activity Tracking:** delega il calcolo del nuovo stato alla funzione `computeStatus` e gestisce la macchina a stati delle sessioni.
- **Posizioni:** salva le coordinate GPS agganciandole all'activity determinata al punto precedente. In caso di falso positivo, salva anche la posizione del pacchetto in hold.
- **Pulizia:** elimina il record da `data_sent_raw` nel blocco `finally`.

4.6.4 Macchina a stati delle attività

Il cuore della logica server-side è una macchina a stati che modella il comportamento dell'animale. Gli stati sono distinti in *attivi* (dispositivo sveglio) e *sleep* (dispositivo in Deep Sleep), con una corrispondenza biunivoca tra le due categorie.

Stato attivo	Stato sleep	Descrizione
i — inside	d	Animale all'interno della geofence
s — search	p	Ricerca attiva: animale fuori zona con allarme
w — walk	z	Animale in passeggiata fuori zona (senza allarme)
v — vehicle/trip	a	Animale su veicolo in movimento

Tabella 1: Macchina a stati delle attività: corrispondenza tra stati attivi e sleep

Tutti gli stati sleep si comportano in modo **uniforme**: vengono sempre chiusi con `is_active = false`. Non esiste più alcun caso speciale per lo stato "a" (trip-sleep). La funzione `computeStatus` normalizza internamente qualsiasi stato sleep nel corrispondente stato attivo prima di qualsiasi valutazione, permettendo di passare direttamente il `closedStatus` della sessione precedente senza pre-elaborazione esterna. Questo garantisce che la logica trip (`steps==0` mantiene "v") funzioni correttamente anche dopo una fase di sleep.

Le transizioni tra stati attivi sono governate dalla funzione `computeStatus` che, ricevuto un pacchetto, calcola il nuovo stato secondo la seguente priorità:

1. Se `trip = true` $\Rightarrow$ stato `"v"` (priorità assoluta, solo dopo conferma del secondo pacchetto consecutivo).
2. Se il dispositivo era in `"v"` (o `"a"`, normalizzato a `"v"`) e `trip` torna `false` con `steps == 0` $\Rightarrow$ rimane `"v"` (ancora fermo dopo il viaggio).
3. Se il dispositivo era in `"v"` e `trip` torna `false` con `steps > 0` $\Rightarrow$ ricalcolo pulito della geofence.
4. Altrimenti $\Rightarrow$ la macchina a stati valuta la posizione rispetto alle geofence e l'eventuale allarme utente: `"i"` se dentro, `"s"` se fuori con allarme attivo, `"w"` se fuori senza allarme o senza geofence configurate.

4.6.5 Gestione del Risveglio e Deduplicazione delle Sessioni

Un aspetto critico della logica è la gestione del risveglio dal Deep Sleep. Quando il dispositivo si sveglia non trova una sessione attiva (quella precedente è stata chiusa allo sleep). Il codice cerca la sessione chiusa più recente (`recentClosed`) per ricavare il `prevStatus` da passare a `computeStatus`: questo è fondamentale per il trip-sleep, dove `"a"` viene normalizzato a `"v"` e con `steps==0` mantiene correttamente lo stato `"v"`.

Il sistema distingue tre scenari:

- **Risveglio nello stesso stato:** se la sessione era stata chiusa in uno stato sleep e il nuovo stato attivo calcolato corrisponde allo stesso stato attivo di partenza, la sessione viene riattivata (`is_active = true`) senza limiti di tempo. Questo permette di collegare correttamente le attività attraverso i cicli sleep, anche dopo ore di inattività.
- **Risveglio con cambio di stato:** la sessione chiusa viene prima corretta riportando il suo status alla versione attiva (es. `"d"` $\rightarrow$ `"i"`, `"z"` $\rightarrow$ `"w"`), così da garantire la correttezza del riepilogo storico. Poi viene aperta una nuova sessione con il nuovo stato calcolato.
- **Deduplicazione micro-interruzioni:** per sessioni chiuse non in stato sleep ma entro una finestra di 120 secondi, con stato compatibile, viene applicata una logica di riattivazione per evitare frammentazione causata da brevi perdite di segnale LTE.

Le sessioni chiuse con `anomaly = true` (chiuse dal watchdog) non vengono mai riattivate: viene sempre creata una nuova sessione, evitando che una sessione anomala venga ripresa come se nulla fosse.

4.6.6 Watchdog di Inattività e Split Notturmo

Il sistema include due *cron job* per garantire la coerenza delle sessioni nel tempo:

- **Watchdog (ogni minuto):** controlla che nessuna sessione rimanga aperta indefinitamente a causa di un'interruzione imprevista del dispositivo (perdita di segnale LTE, spegnimento improvviso). Se una sessione attiva non riceve aggiornamenti per più di 10 minuti, viene chiusa automaticamente impostando `is_active = false` e il flag `anomaly = true`, e viene salvato un evento `watchdog` per tracciabilità. Il watchdog non tocca mai le sessioni in stato sleep, che per design possono rimanere

inattive a lungo. Cancella inoltre il campo `pending_trip` sulla board in caso di chiusura per inattività, evitando falsi positivi al riavvio.

- **Split di mezzanotte (23:59 ora italiana):** gestisce le sessioni che attraversano il cambio di giornata. Per ogni board, l'ultima sessione attiva viene chiusa alle 23:59:59 e ne viene aperta una nuova identica a partire dalle 00:00:00 del giorno successivo. Le sessioni in stato sleep vengono corrette al corrispondente stato attivo prima della chiusura, garantendo la correttezza del riepilogo giornaliero. L'orario è calcolato dinamicamente tramite la funzione `getItalyTime()`, che gestisce automaticamente il cambio tra ora legale (UTC+2) e ora solare (UTC+1) secondo le regole europee, senza offset hardcoded. Il cron di mezzanotte include inoltre una verifica watchdog prioritaria: sessioni anomale (inattive da più di 10 minuti) vengono chiuse direttamente senza essere duplicate nel giorno successivo.

4.6.7 Vantaggi Architetture e Corrispondenza IoTWF

Questa implementazione riflette i principi del Modello IoTWF in due punti chiave:

1. **Efficienza del Livello 3 (Edge):** La LilyGo invia un unico pacchetto “bulk”, riducendo il tempo di attività del modem LTE e preservando la batteria.
2. **Ottimizzazione del Livello 5 (Data Abstraction):** L'applicazione Flutter non deve processare dati grezzi; interroga API già normalizzate dal trigger, garantendo una visualizzazione fluida della cronologia.

5 Sistema di Notifiche Push

L'implementazione di un sistema di notifiche push è fondamentale per un dispositivo di tracciamento animali, in quanto permette di avvisare tempestivamente l'utente in caso di pericoli (uscita dal perimetro sicuro) o problemi tecnici (batteria scarica).

5.1 Analisi dei Vincoli Tecnologici: Incompatibilità Goja e Firebase

Durante la fase di sviluppo, è emerso un vincolo tecnico significativo riguardante l'integrazione tra il backend PocketBase e l'SDK ufficiale di Firebase. PocketBase è scritto in linguaggio Go e utilizza **Goja** come motore di esecuzione JavaScript per la gestione della logica server-side (hooks). Goja è un'implementazione di ECMAScript 5.1 che presenta limitazioni strutturali rispetto a un ambiente Node.js [10] standard:

- **Mancanza di moduli nativi:** Goja non supporta i moduli core di Node.js (come `fs`, `crypto` o `net`), indispensabili per il funzionamento dell'SDK di Firebase.
- **Runtime isolato:** L'SDK `firebase-admin` [11] richiede un ambiente di esecuzione completo per gestire l'autenticazione tramite Service Account e le connessioni persistenti ai server Google.

Per superare queste limitazioni, è stata adottata un'architettura a **Bridge**, separando la logica di business dall'invio fisico dei messaggi.

5.2 Architettura del FCM Bridge

Il sistema si basa su un microservizio esterno sviluppato in Node.js [10], denominato *FCM Bridge*. L'interazione avviene secondo il seguente flusso:

1. **PocketBase (Goja)**: Rileva un evento (es. batteria scarica) e invia una richiesta HTTP POST al bridge tramite il modulo interno `$http.send`.
2. **FCM Bridge (Node.js)**: Riceve la richiesta, carica il Service Account di Firebase e inoltra la notifica tramite l'SDK ufficiale [11].
3. **Feedback e pulizia token**: Il bridge restituisce lo stato della consegna. In caso di errore 404 (token non registrato, app disinstallata) o 400 (token malformato), il backend provvede alla rimozione automatica del token dal profilo utente. I token FCM sono memorizzati come array JSON per supportare più dispositivi per singolo utente.

5.3 Logica di Invio e Trigger

Le notifiche vengono scatenate dai seguenti eventi, ciascuno con logica contestuale per evitare invii ridondanti:

Evento	Condizione di trigger	Nota
Uscita dalla zona	$i \rightarrow s$ oppure $i \rightarrow w$	Dipende dalla presenza di allarme utente
Rientro in zona	$s \rightarrow i$ oppure $w \rightarrow i$	
Allarme da passeggiata	$w \rightarrow s$	Allarme riattivato mentre fuori zona
Inizio viaggio confermato	qualsiasi $\rightarrow v$	Solo dopo il secondo trip=true consecutivo
Fine viaggio in zona	$v \rightarrow i$	Arrivato a destinazione
Fine viaggio fuori zona (allarme)	$v \rightarrow s$	Animale fuggito dal veicolo
Fine viaggio fuori zona (libero)	$v \rightarrow w$	Passeggiata post-viaggio
Nessuna geofence	$i \text{ o } s \rightarrow w$	Zone disattivate
Batteria bassa	$\leq 20\%$ (cambio soglia)	Solo al cambio di fascia
Batteria critica	$\leq 10\%$ (cambio soglia)	Solo al cambio di fascia
Batteria in carica	charging passa a true	Solo al cambio di stato
Watchdog	Silenzio > 10 minuti	Chiusura automatica sessione

Tabella 2: Trigger del sistema di notifiche push

6 Front-end e Applicazione Mobile

L'applicazione mobile è stata sviluppata in Flutter [12], un framework cross-platform che permette di produrre un'unica base di codice per Android e iOS. La comunicazione con il backend avviene tramite il PocketBase SDK ufficiale per Dart, che gestisce internamente le chiamate REST e i meccanismi di autenticazione.

6.1 Architettura a livelli

Il codice è strutturato in quattro livelli principali, per separare le diverse responsabilità:

- **Repositories:** accesso ai dati remoti tramite PocketBase SDK, fornendo alle schermate dati in tempo reale e pronti all'uso.
- **Models:** definiscono la struttura dei dati (es. Utente, Posizione) convertendo i record JSON del backend in oggetti Dart.
- **Services:** gestiscono le funzionalità di sistema e le integrazioni esterne (es. autenticazione, mappe, tracciamento GPS e notifiche).
- **Screens:** interfaccia utente che si limita a richiedere i dati ai repository e a reagire ai loro aggiornamenti.

Questa struttura garantisce un codice ordinato e facile da aggiornare, mantenendo la logica di funzionamento ben separata dall'interfaccia grafica.

6.2 Autenticazione e persistenza della sessione

Per permettere all'utente di restare collegato in modo permanente e sicuro, abbiamo esteso l'`AuthStore` nativo di PocketBase, che normalmente salva i dati solo temporaneamente in memoria. L'autenticazione è quindi gestita tramite un `SecureAuthStore` personalizzato che agisce come una *cassaforte digitale*, riceve il token (JSON Web Token) dal server al momento del login e lo archivia in forma cifrata all'interno del dispositivo tramite `flutter_secure_storage`. Questo sistema sfrutta i sistemi di protezione hardware del dispositivo (Keystore su Android e Keychain su iOS) per impedire accessi non autorizzati.

Al riavvio dell'app, il token viene riletto dallo storage sicuro e viene tentato un refresh automatico della sessione. In caso di successo, l'utente accede direttamente alla schermata principale senza dover reinserire le credenziali; in caso di token scaduto o non valido, lo store viene svuotato e l'utente viene reindirizzato alla schermata di login.

6.3 Splash Screen e Caricamento Parallelo

La schermata di avvio svolge un ruolo attivo nel precaricare i dati necessari alla home page. Una volta verificata l'autenticazione, vengono avviate in parallelo, le seguenti operazioni:

- recupero del profilo utente e dello stato dell'allarme;
- recupero dell'ultima posizione GPS registrata dal dispositivo;
- recupero delle attività del giorno corrente filtrate per `board_id`.

Il `board_id`, identificativo univoco della board LilyGo associata a uno o più utenti, viene risolto dinamicamente interrogando la collezione `boards` sul backend tramite repository. Questo permette all'app di trovare il dispositivo corretto automaticamente, evitando di dover inserire il codice manualmente nel software.

6.4 Comunicazione in Tempo Reale

Completata la fase iniziale di caricamento dati, l'app stabilisce una **connessione persistente** al database, passando da una visualizzazione statica ad un monitoraggio dinamico.

Per le funzionalità che richiedono aggiornamenti continui come la posizione GPS, l'app sfrutta il meccanismo di *subscribe* offerto da PocketBase, basato su **Server-Sent Events**. I repository (come **PositionsRepository**) espongono uno **Stream** tipizzato (ad esempio **Stream<Positions>**) a cui le schermate si sottoscrivono tramite una **StreamSubscription**.

In questo modo, ogni nuovo record inserito nel database viene notificato istantaneamente all'app. L'interfaccia utente si aggiorna dinamicamente evitando la tecnica del *polling*, che costringerebbe l'app a inviare continue richieste al server, ed eliminando la necessità di qualsiasi ricaricamento manuale della pagina da parte dell'utente.

Per garantire l'efficienza dell'applicativo e prevenire *memory leak*, ogni sottoscrizione viene rigorosamente interrotta tramite il comando **cancel()** nel metodo **dispose()** quando l'utente naviga verso un'altra schermata. Questa accortezza evita di mantenere attive connessioni inutili in background, ottimizzando l'uso della batteria e del traffico dati.

6.5 Geofencing e Geocoding

La logica di geofencing è implementata anche dal lato client tramite l'algoritmo *Ray-Casting*. Dato un punto GPS (latitudine, longitudine) e un poligono definito da una lista di vertici, l'algoritmo traccia un raggio orizzontale dal punto verso l'infinito e conta le intersezioni con i lati del poligono, se il numero è dispari indica che il punto è interno al poligono, altrimenti è esterno. Questa scelta progettuale permette di gestire recinti virtuali di qualsiasi forma, adattandosi efficacemente ai confini del mondo reale.

Le zone vengono recuperate dalla collezione **geofences** su PocketBase, dove ogni record memorizza il nome della zona, i dati dell'indirizzo e la lista dei vertici del poligono. Solo le zone con il flag **is_active** impostato a **true** vengono considerate nel calcolo. Il risultato della verifica è una stringa con il nome della zona di appartenenza, oppure *"Fuori zona sicura"* se il punto non rientra in nessuna area attiva.

Per arricchire l'interfaccia, l'applicativo integra un servizio di **geocoding** basato sulle API di *Nominatim* [13]. Quest'ultimo agisce come motore di ricerca che interroga il database cartografico open-source di OpenStreetMap, permettendo di:

- **Reverse Geocoding**: tradurre le coordinate GPS in indirizzi testuali completi (Via, Civico, CAP e Città) per facilitare la lettura all'utente;
- **Forward Geocoding**: consentire all'utente di creare nuove zone inserendo un indirizzo testuale, che il sistema converte automaticamente in coordinate geografiche per posizionare correttamente il poligono sulla mappa.

6.6 Permessi e Notifiche Push

Le notifiche push sono gestite tramite **Firestore Cloud Messaging**. Al primo avvio, l'app mostra un dialog personalizzato che spiega all'utente il motivo della richiesta del permesso, prima di invocare la richiesta di sistema. Una volta ottenuto il consenso, il token FCM del dispositivo viene sincronizzato con il profilo utente su PocketBase, nel campo **tokenFCM** modellato come una lista, permettendo all'utente di ricevere notifiche su più dispositivi contemporaneamente. Prima del salvataggio, un controllo di duplicazione verifica che il token del dispositivo corrente non sia già presente nella lista, evitando ricezioni di notifiche ridondanti.

Analogamente alla gestione delle notifiche, l'app mostra al primo avvio un dialog personalizzato per spiegare le finalità del monitoraggio della posizione dell'utente prima di attivare la richiesta di sistema. Il consenso ottenuto viene poi incrociato con l'effettivo stato dell'antenna GPS tramite un controllo centralizzato, che aggiorna una variabile di stato globale per abilitare il recupero della posizione con alta precisione. Infine, l'utilizzo di `SharedPreferences` permette di memorizzare l'avvenuta visualizzazione dell'informativa, evitando che il pop-up esplicativo appaia ripetutamente ad ogni avvio dell'applicazione.

6.7 Funzionalità Principali

Mappa in Tempo Reale La schermata principale mostra la posizione aggiornata dell'animale su mappa tramite la libreria `flutter_map` con tile OpenStreetMap. La posizione viene aggiornata ad ogni evento ricevuto dallo stream in tempo reale. L'indirizzo corrispondente alle coordinate viene risolto in forma testuale tramite il servizio di reverse geocoding Nominatim.

Riepilogo Giornaliero La schermata di recap, accessibile selezionando una data dal calendario, recupera dal backend le attività e le posizioni GPS relative a quel giorno. Le posizioni vengono rappresentate come una polilinea sulla mappa, restituendo una visualizzazione del percorso effettuato dall'animale. Le statistiche aggregate (passi totali, distanza stimata in chilometri e durata complessiva del movimento) vengono calcolate client-side a partire dai dati ricevuti.

Stato della batteria La schermata dedicata mostra la percentuale di carica residua tramite un indicatore circolare animato con gradiente. Quando il campo `charging` del record è impostato a `true`, l'indicatore entra in modalità animazione rotante per segnalare visivamente la ricarica in corso. La durata stimata residua viene calcolata proporzionalmente alla capacità nominale della batteria (3000 mAh).

Gestione delle Zone Sicure L'app consente di creare, modificare e disattivare le zone di geofencing tramite un editor poligonale interattivo su mappa. L'utente può disegnare il perimetro della zona toccando i vertici direttamente sulla mappa; al salvataggio, le coordinate dei vertici e il centroide del poligono vengono persistiti su PocketBase. L'indirizzo associato alla zona viene ricavato automaticamente tramite geocoding inverso a partire dal centroide.

7 Test e Risultati

8 Limiti del Sistema

Nonostante i risultati ottenuti, il sistema presenta alcune limitazioni:

- **Dipendenza dalla rete LTE:** in assenza di copertura, la trasmissione dei dati non è garantita.
- **Prestazioni indoor:** il modulo GNSS risulta poco affidabile in ambienti chiusi.
- **Backend locale:** l'utilizzo di PocketBase su macchina locale, esposto tramite ngrok, introduce dipendenze da servizi esterni e limita la scalabilità.

- **Consumo energetico del modem LTE:** la trasmissione dati rappresenta la principale fonte di consumo.
- **Race condition sui pacchetti simultanei:** in caso di ricezione quasi-simultanea di due pacchetti dallo stesso dispositivo, entrambi potrebbero leggere `pending_trip = null` prima che il primo abbia completato la scrittura, impedendo la corretta conferma del viaggio. PocketBase non espone transazioni atomiche sui record, rendendo questo scenario non eliminabile per via software; in pratica la frequenza di invio dei pacchetti rende l'evento raro.

9 Conclusioni

Riferimenti bibliografici

- [1] *T-SIM7670G-S3 Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/docs/en/esp32s3/sim7670g-s3/REAMDE.MD>.
- [2] *LTE Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/LTE%20Antenna%20Specifications.pdf>.
- [3] *GPS Antenna Specifications*. URL: <https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/GPS%20Antenna%20Specifications.pdf>.
- [4] *BMA400 CJMCU-400 Datasheet*. URL: https://www.bosch-sensortec.com/media/boschsensortec/downloads/product_flyer/bst-bma400-fl000.pdf.
- [5] *PocketBase Documentation*. URL: <https://pocketbase.io/docs/>.
- [6] *ngrok Documentation - HTTP Tunnels*. URL: <https://ngrok.com/docs/secure-tunnels/>.
- [7] *rclone - rsync for cloud storage*. URL: <https://rclone.org/>.
- [8] *Backblaze B2 Cloud Storage*. URL: <https://www.backblaze.com/b2/cloud-storage.html>.
- [9] *UptimeRobot - Free Website Monitoring Service*. URL: <https://uptimerobot.com/>.
- [10] *Node.js Documentation*. URL: <https://nodejs.org/en/docs/>.
- [11] *Firebase Admin SDK - Cloud Messaging*. URL: <https://firebase.google.com/docs/cloud-messaging/server>.
- [12] *Flutter Documentation*. URL: <https://docs.flutter.dev/>.
- [13] *Nominatim Documentation*. URL: <https://nominatim.org/release-docs/latest/>.